

CKA Intensive

Day 1 — Foundations, Architecture, Workloads, Config

Luis Torres

4-day intensive · target: **Kubernetes v1.32**

Today (eight blocks)

1. Welcome + exam logistics
2. Kubernetes architecture (deep)
3. The Kubernetes API + reconciliation model
4. **Lab 0** — environment setup
5. kubectl mastery
6. **Lab 1** — kubectl basics
7. Workload controllers (Pod → Deployment → DaemonSet → StatefulSet → Job → CronJob)
8. **Lab 2** — workloads
9. **ConfigMaps, Secrets, Probes, init containers**
10. **Lab 2b** — config + probes

The exam at a glance

- **2 hours**, performance-based — no multiple choice
- ~15–20 hands-on tasks, each weighted by %
- **Pass: 66%** (lowered from 74% in 2024)
- Cost: ~\$395 USD, includes **1 free retake** within 12 months
- **2 free killer.sh sessions** (harder than the real exam — use both)
- Allowed docs: `kubernetes.io/docs`, `kubernetes.io/blog`, `helm.sh/docs`
- Browser-based, **PSI proctored**, **Chrome only**
- Result in ~24h by email
- Digital badge on Credly, valid **2 years**

Curriculum weights (2024 refresh, still current)

Domain	Weight
Cluster Architecture, Install & Config	25%
Workloads & Scheduling	15%
Services & Networking	20%
Storage	10%
Troubleshooting	30%

Implication: Troubleshooting is the biggest single category — Day 4 spends most of itself there. But every domain *a*lso contains troubleshooting in disguise.

What the 2024 refresh added vs old curriculum

If you've seen older CKA prep material — these are new and **on the exam**:

- **Helm + Kustomize** (cluster architecture)
- **Gateway API** alongside Ingress (networking)
- **HPA / autoscaling** (workloads)
- **CRDs and operators** (cluster architecture, conceptual)
- **Dynamic volume provisioning** is now explicitly emphasized

We cover all of these. Older course material that skips them will leave you a half-letter-grade short.

What Kubernetes is (in one sentence)

A cluster API for declaring the **desired state** of containerized workloads, with **controllers** that reconcile actual state to match.

Three ideas that explain almost everything:

1. **Declarative** — you describe *what*, controllers figure out *how*.
2. **Eventually consistent** — there is *always* skew between what you asked for and what's running. The system converges.
3. **Everything is an object** — every kind has the same lifecycle: create, get, watch, update, delete.

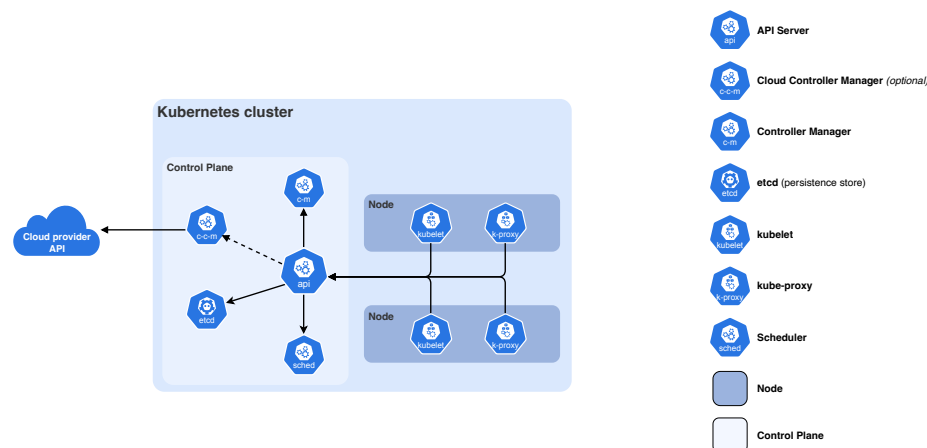
Architecture — the whole picture

Control plane (one set per cluster, typically on dedicated nodes):

- kube-apiserver
- etcd
- kube-scheduler
- kube-controller-manager
- cloud-controller-manager (*optional*)

Every node (control plane and worker):

- kubelet
- kube-proxy
- container runtime (containerd/CRI-O)
- CNI plugin



kube-apiserver

The **only** thing anyone or anything talks to. Everything else is a client.

- REST API over HTTPS (:6443)
- Authn (certs, tokens, OIDC, webhook) → Authz (RBAC, Node, Webhook) → Admission (mutating + validating) → etcd
- **Watches:** clients open a long-lived HTTP stream; apiserver pushes object changes
- Stateless — scale horizontally; etcd is the only stateful piece

The watch/list/reconcile loop is *the* Kubernetes pattern. Every controller is one.

etcd

A distributed key/value store using **Raft** consensus.

- All cluster state lives here. Lose etcd → lose the cluster.
- Runs as a **static pod** on control-plane nodes by default (kubeadm)
- **Listens on** `:2379` (clients) and `:2380` (peer-to-peer)
- TLS-protected, certs in `/etc/kubernetes/pki/etcd/`
- Recommended: odd number of members (3 or 5) for quorum

You will back it up and restore it on Day 4. Memorize the commands.

kube-scheduler

Watches for **pods without** `nodeName` and assigns one.

Two phases:

1. **Filter** — which nodes are eligible? (taints, affinity, port conflicts, resource requests)
2. **Score** — among eligible, which is best? (spread, image locality, etc.)

Picks the highest-scoring node and writes `pod.spec.nodeName` back to etcd via apiserver.

Doesn't actually start the pod — that's kubelet's job.

kube-controller-manager

A single binary running **many** built-in controllers as goroutines:

- Deployment controller, ReplicaSet controller, DaemonSet, StatefulSet, Job, CronJob
- Node controller (mark NotReady), endpoint(slice) controller, namespace, SA, GC, ...

Each one is a watch loop: *observe* → *diff desired vs actual* → *take action*.

The reconciliation pattern is identical for built-in and custom controllers. Once you understand one, you understand all of them.

cloud-controller-manager

Talks to your cloud provider's API.

- LoadBalancer Services → provision an actual cloud LB
- Node lifecycle (was this VM deleted? mark Node deleted)
- Routes (where needed)

Only present when you're on a cloud (EKS/GKE/AKS/etc.). On kind/kubeadm-on-bare-metal, absent — which is why **LoadBalancer** Services stay **Pending** on kind unless you install MetalLB or cloud-provider-kind.

kubelet

The agent on every node. The control plane's hands and eyes.

- Watches apiserver for **pods assigned to this node**
- Talks to the **CRI runtime** to pull images and run containers
- Reports node status + pod status back to apiserver
- Runs **probes** (liveness/readiness/startup)
- Mounts volumes for pods
- Reads **static pod manifests** from `/etc/kubernetes/manifests/` — that's how control-plane pods themselves run

If kubelet stops: node goes `NotReady`. Existing pods keep running until the kernel reaps them.

kube-proxy

Programs the **node's networking** so Service VIPs work.

Three modes (1.32):

- **iptables** — default; rules per Service+endpoint; $O(n)$ traversal
- **IPVS** — kernel-level load balancer; better at high scale
- **nftables** — GA in 1.32; modern replacement for iptables mode

Without kube-proxy, you can still create Services, but the VIP won't route anywhere. Some CNIs (Cilium with eBPF) replace kube-proxy entirely.

Container Runtime (CRI)

Kubernetes doesn't run containers — it asks a CRI-compliant runtime to.

- **containerd** (most common on managed clouds and kubeadm)
- **CRI-O** (Red Hat / OKD)
- **✗ Docker is gone** — dockershim removed in v1.24

crictl is your debugging tool on a node (similar to **docker** CLI, but talks to CRI):

```
sudo crictl ps -a           # list containers
sudo crictl logs <id>
sudo crictl inspect <id>
sudo crictl pods            # list pod sandboxes
```

CNI (Container Network Interface)

The plugin that gives **pods their IPs and connectivity**.

CNI	Strengths
Calico	Mature; NetworkPolicy enforcement; BGP; eBPF mode
Cilium	eBPF-native; kube-proxy replacement; Hubble observability; Gateway API
Flannel	Simple; no policy
Weave	Maintenance has slowed
kindnet	Default in <code>kind</code> ; does NOT enforce NetworkPolicy

Without a CNI, nodes are `NotReady` and pods can't get IPs.

How a Pod gets created — full path

1. `kubectl apply -f pod.yaml`
2. **apiserver**: authn → authz → admission → write to **etcd**
3. **scheduler** watches unscheduled pods, runs filter+score, writes `nodeName` via apiserver
4. **kubelet** on that node watches its own pods, sees the new assignment
5. kubelet asks **CRI runtime** to pull image and start container(s)
6. kubelet sets up volumes, networks (via CNI), then runs containers
7. kubelet reports `Running` + probe results back to apiserver
8. (If part of a Service) **endpoint(slice) controller** adds pod IP to endpoints
9. **kube-proxy** on every node updates iptables/IPVS rules

Steps 1–3 are seconds. Step 5 dominates wall-clock (image pull).

The mental model: watch / list / reconcile

Every controller (and most user code in operators) does this loop:



- "List once, watch thereafter" — efficient state synchronization
- **Level-triggered, not edge-triggered:** if you miss an event, the next reconcile fixes it
- Idempotent: applying the same desired state twice is a no-op

Kubernetes objects — anatomy

```
apiVersion: apps/v1           # group/version
kind: Deployment              # kind
metadata:
  name: web
  namespace: prod
  labels: { app: web, tier: frontend }
  annotations: { team: platform }
spec:                         # desired state – you write this
  ...
status:                       # actual state – controllers write this
  ...
```

- **Group/version/kind (GVK)** identifies the schema
- `metadata.name` is unique within `namespace` per kind
- You write `spec`. You read `status`. Don't put your state in `metadata`.

Namespaces

A namespace is a **scope** for naming objects, plus a unit for:

- RBAC (per-namespace roles)
- ResourceQuota / LimitRange
- NetworkPolicy
- DNS suffix (`<svc>.<ns>.svc.cluster.local`)

Not in any namespace: nodes, PVs, ClusterRoles, StorageClasses, CRDs, the namespace object itself.

```
k api-resources --namespaced=true  
k api-resources --namespaced=false
```

Lab 0 — Environment

→ `trainees/day1/labs/lab0-environment.md`

60 min. Verify Docker/Podman, kubectl 1.32, kind, helm 3.16, vim. Bring up `kind-3node` cluster on v1.32. Verify `kubectl get nodes` shows 3 Ready nodes.

If you can't get past Lab 0 by the end of the slot, tell the trainer — we don't continue with a broken setup.

kubectl — your only friend on exam day

```
alias k=kubectl
source <(kubectl completion bash)      # zsh: kubectl completion zsh
complete -o default -F __start_kubectl k

export do='--dry-run=client -o yaml'   # generate YAML
export now='--grace-period=0 --force'  # force delete
```

Set this in **EVERY** new terminal on the exam. They give you a `~/.bashrc` snippet block — paste it.

You don't get tab completion for `k` until you `complete -F __start_kubectl k`.

Imperative vs declarative

```
# Imperative – fast, exam-friendly
k run nginx --image=nginx:1.27
k create deploy web --image=nginx:1.27 --replicas=3
k expose deploy web --port=80

# Declarative – production / git-tracked
k apply -f deployment.yaml
```

The pragmatic exam workflow:

```
k create deploy web --image=nginx:1.27 $do > web.yaml
vi web.yaml          # add what generator can't (volumes, probes, ...)
k apply -f web.yaml
```

Generators you'll use on the exam

```
# pod
k run nginx --image=nginx:1.27 $do > pod.yaml

# deployment
k create deploy web --image=nginx:1.27 --replicas=3 --port=80 $do > dep.yaml

# service
k expose deploy web --port=80 --target-port=80 --name=web $do > svc.yaml
k create svc clusterip web --tcp=80:80 $do > svc.yaml

# configmap / secret
k create cm app --from-literal=KEY=val $do > cm.yaml
k create secret generic db --from-literal=password=p $do > sec.yaml

# job / cronjob
k create job hello --image=busybox -- echo hi $do > job.yaml
k create cronjob c --image=busybox --schedule="*/1 * * * *" -- echo hi $do > cj.yaml

# rbac
k create sa dev $do > sa.yaml
k create role r --verb=get,list --resource=pods $do > role.yaml
k create rolebinding rb --role=r --serviceaccount=ns:dev $do > rb.yaml

# resourcequota
k create quota nq --hard=cpu=2,memory=1Gi,pods=5 $do > q.yaml
```


The 80% of kubectl

Verb	What it does
<code>get</code>	list objects (add <code>-w</code> to watch, <code>-o wide</code> , <code>-o yaml</code> , <code>-o jsonpath=...</code>)
<code>describe</code>	human-readable detail; always check Events at the bottom
<code>logs</code>	container logs (<code>-f</code> , <code>--previous</code> , <code>-c <container></code> , <code>-l <selector></code>)
<code>exec</code>	run command in container (<code>-it -- sh</code>)
<code>apply</code>	declarative create/update
<code>create</code> , <code>run</code> , <code>expose</code>	imperative shortcuts (+ generators with <code>\$do</code>)
<code>edit</code>	open in <code>\$EDITOR</code> ; saves on <code>:wq</code>
<code>patch</code>	scripted partial update; better than <code>edit</code> for known fields
<code>delete</code>	remove (+ <code>\$now</code> for stuck objects)
<code>explain</code>	schema docs; <code>--recursive</code> to see all fields
<code>rollout status/history/undo</code>	manage Deployment rollouts

kubectl explain — your offline docs

```
k explain deployment.spec.strategy --recursive
k explain pod.spec.containers.livenessProbe
k explain networkpolicy.spec.ingress
```

Beats hunting through **kubernetes.io/docs** for field names. **No internet round-trip on the exam.**

For verb-level help:

```
k create -h | less
k expose -h | less
```

Read the examples in **-h** — they're better than half the doc pages.

Output formats: `-o jsonpath` and `-o custom-columns`

When `get` isn't enough:

```
k get pod web -o jsonpath='{.status.podIP}'  
k get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.phase}{"\n"}{end}'  
  
k get nodes -o custom-columns=NAME:.metadata.name,VER:.status.nodeInfo.kubeletVersion  
  
k get pods --sort-by=.metadata.creationTimestamp  
k get pods --field-selector=status.phase=Pending -A
```

Worth memorizing one or two patterns for exam questions like *"export the pod name running the highest version of nginx."*

Contexts and namespaces

```
k config get-contexts
k config use-context <ctx>
k config set-context --current --namespace=<ns>
k config view --minify # what am I pointed at right now?
```

Every exam question states the context and namespace. Set both before reading the rest of the question. Forgetting to switch costs you the whole question.

Lab 1 — kubectl basics

→ `trainees/day1/labs/lab1-kubectl-basics.md`

60 min. Contexts, imperative, generators, deployments, expose, jsonpath, sort.

The point is **muscle memory**, not understanding. Type, don't paste.

Workload controllers — the family tree

```
Pod          ← smallest schedulable unit
|
ReplicaSet   ← keeps N pods running (rarely written directly)
|
Deployment   ← rolling updates + rollback history
|
DaemonSet    ← one pod per (selected) node
|
StatefulSet  ← stable identity + storage, ordered
|
Job          ← run to completion (success/failure)
CronJob      ← scheduled Jobs
```

Each controller is one watch/reconcile loop. They differ only in *what they reconcile toward*.

Pod — the atom

A pod is **one or more containers** that share:

- a Linux network namespace (same IP, same port space)
- IPC namespace
- volumes
- (optionally) PID namespace via `shareProcessNamespace`

One main container per pod is the norm. Sidecars (log shipper, proxy, fetcher) are valid; init containers are common.

Pods are **mortal**: they are recreated, never resurrected. New pod = new IP. Don't write code that depends on pod IPs.

Minimal Pod YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels: { app: web }
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports: [{ containerPort: 80 }]
    resources:
      requests: { cpu: 100m, memory: 128Mi }
      limits: { cpu: 500m, memory: 256Mi }
```

You'll write this YAML in your sleep. Memorize the indentation.

Multi-container pod (sidecar pattern)

```
spec:
  containers:
    - name: app
      image: myapp:1.0
      volumeMounts: [{ name: logs, mountPath: /var/log/app }]
    - name: log-shipper
      image: fluent-bit:3.0
      volumeMounts: [{ name: logs, mountPath: /var/log/app, readOnly: true }]
  volumes:
    - name: logs
      emptyDir: {}
```

The two containers share `logs` via `emptyDir`. Use cases: log shipping, file conversion, service mesh sidecar.

Native sidecar containers (init container with `restartPolicy: Always`) are GA in 1.29 — they start before regular containers and shut down after.

Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports: [{ containerPort: 80 }]
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
```



selector.matchLabels MUST equal **template.metadata.labels** (since 1.16, API rejects otherwise).

Rolling update mechanics

```
k set image deploy/web nginx=nginx:1.28
k rollout status deploy/web
k rollout history deploy/web
k rollout undo deploy/web --to-revision=2
k rollout pause deploy/web
k rollout resume deploy/web
```

Strategy knobs:

Knob	Effect
<code>maxSurge: 25%</code>	up to 25% over desired during rollout
<code>maxUnavailable: 0</code>	never drop below desired — slower, zero downtime
<code>maxUnavailable: 25%</code>	up to 25% under desired — faster, brief degradation
<code>type: Recreate</code>	kill all, then start new — downtime guaranteed

DaemonSet

One pod per **node** (or per matching subset).

```
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: node-watch }
spec:
  selector: { matchLabels: { app: nw } }
  template:
    metadata: { labels: { app: nw } }
    spec:
      tolerations:
        - operator: Exists                # tolerate all taints; needed for cp nodes
      containers:
        - name: agent
          image: busybox:1.36
          command: ["sleep", "86400"]
```

Use for: log shippers, node exporters, CNI agents, kube-proxy itself, CSI drivers.

No **replicas** field. No **strategy** (rolling update via **updateStrategy**).

StatefulSet

For workloads that need:

- **Stable hostnames:** `db-0`, `db-1`, `db-2` — *never* renumbered
- **Stable per-pod storage:** each pod gets its own PVC via `volumeClaimTemplates`
- **Ordered** rollout, scaling, deletion (0 → 1 → 2; reverse on delete)

Requires a **headless Service** (`clusterIP: None`).

Used for: databases (Postgres, MySQL, Mongo), queues (Kafka, RabbitMQ), distributed caches, anything where pod identity matters.

StatefulSet YAML

```
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: db }
spec:
  serviceName: db # headless Service name
  replicas: 3
  selector: { matchLabels: { app: db } }
  template:
    metadata: { labels: { app: db } }
    spec:
      containers:
        - name: db
          image: postgres:16
          ports: [{ containerPort: 5432 }]
          volumeMounts:
            - { name: data, mountPath: /var/lib/postgresql/data }
  volumeClaimTemplates:
    - metadata: { name: data }
      spec:
        accessModes: [ReadWriteOnce]
        resources: { requests: { storage: 5Gi } }
```

Job & CronJob

```
apiVersion: batch/v1
kind: Job
metadata: { name: pi }
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(200)"]
      backoffLimit: 4
      completions: 1
      parallelism: 1
```

```
k create cronjob hello --image=busybox \
  --schedule="*/1 * * * *" -- /bin/sh -c 'echo hello at $(date)'
```

CronJob spawns Jobs on a schedule (UTC by default; `timeZone` field in 1.27+ for explicit TZ).

Lab 2 — Workloads

→ `trainees/day1/labs/lab2-workloads.md`

60 min. Deployment with rolling update + rollback, DaemonSet, StatefulSet, Job, CronJob.

ConfigMap — non-secret config

```
apiVersion: v1
kind: ConfigMap
metadata: { name: app-config }
data:
  LOG_LEVEL: debug
  TIMEOUT: "30s"
  app.properties: |
    foo=bar
    baz=qux
```

Three ways to consume in a Pod:

1. **Env var by key:** `env: [{ name: LOG_LEVEL, valueFrom: { configMapKeyRef: { name: app-config, key: LOG_LEVEL } } }]`
2. **All keys as env vars:** `envFrom: [{ configMapRef: { name: app-config } }]`
3. **Mount as volume:** each key → file at `mountPath/<key>`

ConfigMap consumed three ways — full example

```
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sleep", "3600"]
      env:
        - name: LOG_LEVEL
          valueFrom: { configMapKeyRef: { name: app-config, key: LOG_LEVEL } }
      envFrom:
        - configMapRef: { name: app-config }      # all keys as env
      volumeMounts:
        - { name: cfg, mountPath: /etc/app }
  volumes:
    - name: cfg
      configMap:
        name: app-config
        items:                                     # optional: project specific keys
          - { key: app.properties, path: app.properties }
```

ConfigMap — refresh semantics

Consumption	Refreshes when CM changes?
<code>env</code> (single key)	✗ Never. Restart the pod.
<code>envFrom</code>	✗ Never. Restart the pod.
<code>volumeMount</code> (no subPath)	✓ Yes — kubelet syncs ~every 60s
<code>volumeMount</code> with <code>subPath:</code>	✗ Never
ConfigMap with <code>immutable: true</code>	n/a — can't be edited

Immutable ConfigMaps are a perf optimization at scale (apiserver doesn't have to watch them). Set once:

```
data: { ... }  
immutable: true
```

Secret — same shape, different storage rules

```
apiVersion: v1
kind: Secret
metadata: { name: db }
type: Opaque                                # generic; other types: kubernetes.io/tls, dockerconfigjson, ...
data:                                       # base64-encoded
  password: c2VjcmV0
stringData:                               # plain text; encoded automatically on write
  username: admin
```

Same three consumption patterns as ConfigMap. **Secret is not encrypted at rest by default** — etcd holds base64. Enable encryption at rest in the apiserver config (`--encryption-provider-config`).

Imperative shortcut:

```
k create secret generic db \
  --from-literal=username=admin \
  --from-literal=password=secret
```

Secret types you might see on the exam

Type	Use
Opaque	generic key/value (default)
kubernetes.io/tls	TLS cert+key; for Ingress, webhooks
kubernetes.io/dockerconfigjson	image pull from private registry (<code>imagePullSecrets</code>)
kubernetes.io/service-account-token	(legacy) auto-mounted SA tokens — projected tokens are the modern path
kubernetes.io/basic-auth, ssh-auth	rarely used directly

```
k create secret tls web-tls --cert=tls.crt --key=tls.key
k create secret docker-registry regcred --docker-server=... --docker-username=... --docker-password=...
```

Probes — the three kinds

Probe	Kubelet asks	Failure ⇒
liveness	"Should I still be alive?"	Container is restarted
readiness	"Should I receive traffic?"	Pod is removed from Service endpoints (still running)
startup	"Have I finished booting yet?"	Acts like liveness while it runs; once one success, it deactivates and liveness/readiness take over

Startup exists to handle slow boots without giving liveness a permanent grace period.

Probe handlers — three options

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
    httpHeaders:
      - { name: X-Probe, value: liveness }

readinessProbe:
  tcpSocket:
    port: 5432

startupProbe:
  exec:
    command: ["sh", "-c", "cat /tmp/ready"]
```

Tunables on each:

```
initialDelaySeconds: 0      # wait this long before first probe
periodSeconds: 10          # probe every N seconds
timeoutSeconds: 1          # each probe must respond within
successThreshold: 1         # for readiness only (back to ready after N successes)
failureThreshold: 3         # mark failed after N consecutive failures
```

Probe pitfalls

- **Liveness too aggressive** → restart loops on slow startup. *Fix*: startup probe.
- **No readiness probe** → traffic hits a pod that's not actually ready (still warming caches). Result: bursts of 5xx after rollouts.
- **Liveness == readiness with same command** → during cold start, *all* replicas fail liveness simultaneously, and the Deployment kills them all.
- **exec probe with heavy command** → eats CPU + has fork/exec overhead. Prefer **httpGet** when possible.
- **Probe pointing at a different container's port** in a multi-container pod — pods don't share ports semantically per container, but the network namespace IS shared. Be explicit.

init containers

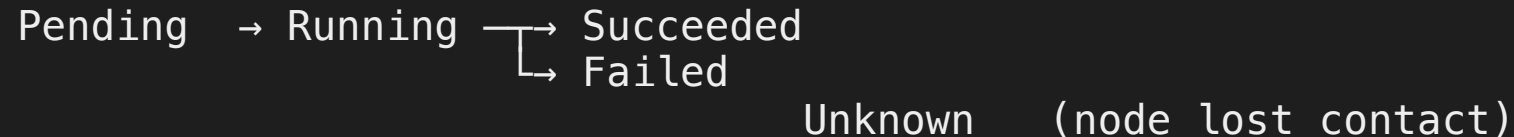
Run **before** regular containers, **in order**, each to completion.

```
spec:
  initContainers:
    - name: wait-db
      image: busybox:1.36
      command: ["sh", "-c", "until nc -z db 5432; do sleep 2; done"]
    - name: migrate
      image: myapp/migrator:1.0
  containers:
    - name: app
      image: myapp:1.0
```

If any init container fails, the pod restarts the init chain.

Use cases: wait for a dependency, fetch config from a vault, run a migration, set kernel sysctls (privileged init).

Pod lifecycle states



Pending substates (in `describe`):

- `ContainerCreating` — image pulling, volume mounting
- `ImagePullBackOff` — image name/registry/creds problem
- `CreateContainerConfigError` — referenced ConfigMap/Secret doesn't exist
- `CrashLoopBackOff` — container started then exited; backoff doubles each retry

```
k get pods -w
k describe pod <p>          # see events at the bottom
k logs <p> --previous        # logs from the previous crash
```

QoS classes (preview — full coverage Day 3)

The kubelet assigns a QoS class to each pod, based on requests/limits:

Class	Requirement	Eviction priority
Guaranteed	every container has <code>requests == limits</code> for CPU AND memory	last to be evicted
Burstable	at least one container has requests, but not Guaranteed	middle
BestEffort	no requests or limits set	first to be evicted under pressure

Production rule of thumb: set both. Guaranteed for latency-sensitive, Burstable for everything else.

Lab 2b — ConfigMaps, Secrets, Probes

→ `trainees/day1/labs/lab2b-config-probes.md`

45 min. Inject a ConfigMap as env + as volume. Inject a Secret as env. Add liveness + readiness + startup probes to a deployment. Watch what happens when each fails.

Day 1 wrap-up

You can now:

- Articulate what every control-plane and node component does
- Bring up a 3-node cluster on **v1.32** with kind
- Use kubectl fluently (completion + `$do` + `$now`)
- Generate YAML for every workload type imperatively, then edit it
- Inject config via ConfigMap and Secret, three ways
- Wire all three probes correctly

Tonight (45 min):

- One killercode.com free scenario
- Re-do Lab 1 from memory, no notes — time yourself

Tomorrow: Services, DNS, Ingress, Gateway API, NetworkPolicy.

Sleep eight hours.